ACADEMIC RESEARCH PRESENTATION & DEFENSE

Byte-Level Mamba vs. Transformer for Amharic with Brain-Inspired Continual Learning

Presenter & Author: Beknan Chemeda | Project: Data- & Compute-Efficient Generative AI

Living Neural Agent: Hayyuu | Hardware: Nvidia GeForce RTX 3090 (24GB VRAM)

Open-Source Repository: https://github.com/BekiChemeda/ML-Research

EXECUTIVE OVERVIEW

Executive Summary: The 3 Core Research Breakthroughs

1. Token-Free Byte Modeling

- Eliminated the subword tokenizer entirely by training directly on raw UTF-8 bytes (V=256).
- Completely removed the 3.5x 'Token Tax' on Ethiopian languages.
- Reclaimed 97.7% of model parameters for deep neural reasoning.

2. Linear Mamba SSM Victory

- Replaced quadratic Transformer self-attention $O(L^2)$ with linear Selective State Spaces $O(L)$.
- TinyMamba (2.73M) outperformed a 13.04M Transformer (1.322 vs 1.579 BPB) on 1.46GB Amharic text.
- Mamba-Base (26.24M) achieved a new frontier record of 1.0613 BPB!

3. Zero-Forgetting Brain Memory

- Engineered a bio-inspired Complementary Learning System (CLS) for live deployment.
- 1-shot live factual learning in 0.01 seconds via Hebbian fast weights.
- **100% grammar preservation and permanent memory via 20x Sharp-Wave Ripple sleep replay.**

PROBLEM 1: DATA SCARCITY

The Reality of Amharic AI: The Low-Resource Data Ceiling

The Digital Data Scarcity Barrier

• English language AI models are trained on trillions of web tokens (Common Crawl, The Pile).

• Amharic is spoken by >50 million people, but has fewer than 500 million clean digital tokens in the entire world (Andersland, 2024).

• Brute-force data scaling is physically impossible—the text simply does not exist online.

• Low-resource African AI requires extreme architectural efficiency, not massive data hunger.

Our Curated Native Corpus (1.465 GB)

• **Collected 100% human-authored Amharic text from verified sources:**

• Amharic Wikipedia: 21.4 MB (encyclopedic domains)

• MasakhaNews: 9.5 MB (contemporary journalism)

• XL-Sum: 33.6 MB (syntactic summaries)

• GlotCC & C4 Amharic Stream: 1.40 GB clean text

• Strict 30% Ge'ez character ratio filter to remove foreign web noise.

DATA QUALITY INTEGRITY

Why Machine Translation Fails for Low-Resource AI

❌ The Synthetic Translation Trap (Azime et al., 2024)

- Many low-resource AI attempts use Google Translate to convert English datasets into Amharic.
- Machine-translated text suffers from translationese: unnatural sentence structures, lost honorifics, and morphological butchery.
- Training LLMs on translated data leads to hallucination and grammatical degeneration.
- **Scientific Rule: We enforced a 100% native, human-authored data curation policy.**

✓ Strict Filtering & Deduplication Pipeline

- 1. Non-printable Byte Removal: Stripped binary control artifacts.
- 2. Length Thresholding: Discarded snippets under 50 characters.
- 3. Ge'ez Script Density Filter: Ensured >= 30% characters reside in Unicode block U+1200 - U+137F.
- 4. MD5 Deduplication: Eliminated exact document duplicates.
- Result: 1,465,682,927 bytes of pristine native training text.

AMHARIC LINGUISTICS

The Linguistic Challenge: Non-Concatenative Ge'ez Script

Unicode & Byte Structure

• Amharic uses the Ge'ez abugida script (Unicode range U+1200 to U+137F).

• In standard UTF-8 encoding, every Ge'ez character occupies exactly 3 raw bytes (values between 0 and 255).

• Example: Letter 'ኢ' = [225, 138, 148].

• Example: Letter 'ት' = [225, 138, 173].

• The word 'ኢትዮጵያ' contains 6 characters but expands to 18 UTF-8 bytes.

Root-and-Pattern Semitic Morphology

• Amharic is a Semitic language built on non-concatenative 3-consonant roots (e.g. S-B-R = break).

• Words change internally through vowel patterns: ሰበረ (he broke), ተሰበረ (was broken), ስብርባሪ (fragments).

• Standard subword tokenizers slice words left-to-right, splitting root morphemes and creating semantic havoc.

• Byte modeling allows local 1D convolutions to capture sub-character roots dynamically!

TOKENIZATION MECHANICS

The Tokenizer: The Flawed Middleman of Modern AI

How Subword Tokenizers Work (BPE / SentencePiece)

- Modern LLMs (GPT-4, LLaMA) cannot read characters directly.
- They use statistical algorithms (Byte-Pair Encoding) to merge frequent character pairs into vocabulary tokens.
- Designed for English: 1 English word ≈ 1 token (e.g. 'computer' = 1 token).
- Assumes high resource availability where every subword appears millions of times.

Why Tokenizers Break on Low-Resource Languages

- Tokenizers are heavily Anglo-centric: English vocabulary dominates 95%+ of token tables.
- Amharic text is severely fragmented into rare, arbitrary byte-pair merges.
- Causes Out-of-Vocabulary (OOV) errors, phonetic distortion, and massive sequence inflation.
- Core Question: Why not bypass the tokenizer entirely and let AI read raw bytes?

THE TOKEN TAX

Problem 1: The Token Tax & 3.5x Fertility Ratio

Defining the Fertility Ratio

- Fertility Ratio = Total Tokens Generated / Total Words.
- In English: Fertility ≈ 1.0 to 1.2 tokens per word.
- In Amharic (SentencePiece 16k): Fertility ≈ 3.5 tokens per word!
- A single Amharic word like 'እንደምንዋላችሁ' is shattered into 5 to 7 tokens.
- African languages pay a severe computational tax for the same semantic meaning.

Real-World Consequences of the Token Tax

- 1. 3.5x Higher Compute: Requires 3.5x more FLOPs to process the same text.
- 2. 3.5x Context Window Shrinkage: An 8k token window holds only 2.2k Amharic words.
- 3. 3.5x Higher API Costs: Ethiopian users pay 3.5x more money per prompt on commercial LLM APIs.
- 4. 3.5x Inference Latency: Generation is 3.5x slower per sentence.

PARAMETER INEFFICIENCY

Problem 2: The Parameter Allocation Tax (63% Wasted)

The Embedding Table Math

- In any model, the embedding matrix has size: V * d_model.
- For a 32k SentencePiece tokenizer with d_model=256:
- Embedding Parameters = 32,000 * 256 = 8,192,000 weights (8.19 Million!).
- In a 13.04M parameter Transformer, 62.8% of all parameters are trapped in static vocabulary lookup tables!
- Only 37.2% of the model is left for actual reasoning and grammar!

The Byte-Level Liberation (V=256)

- For a Byte-Level model with V=256:
- Embedding Parameters = 256 * 256 = 65,536 weights (0.065 Million).
- The embedding matrix consumes under 2.3% of the parameter budget!
- 97.7% of all parameters are dedicated to deep Mamba sequence layers, state-space recurrence, and intelligence!

TRANSFORMER SCALING LIMITS

Problem 3: The Quadratic Attention Bottleneck $O(L^2)$

Why Transformers Choke on Long Byte Streams

- Because Amharic byte sequences are ~3.5x longer, sequence length (L) increases significantly.
- Transformer Self-Attention computes an L x L attention matrix: $A = Q K^T$.
- Compute and memory scale quadratically: $O(L^2)$.
- If sequence length quadruples (L = 1,000 -> 4,000 bytes), attention calculations explode by 16x (16 Million cells!).

The Generative KV-Cache Memory Trap

- During text generation, Transformers must cache Key and Value states for every previous token.
- KV-cache memory grows linearly with generation length for every layer and attention head.
- On edge hardware or single consumer GPUs, byte-level Transformers suffer Out-of-Memory (OOM) crashes.
- Conclusion: Transformers are structurally incompatible with byte-level sequences!

STABILITY-PLASTICITY DILEMMA

Problem 4: Catastrophic Forgetting in Continual Learning

The Failure of Online Fine-Tuning

• In real-world deployment (e.g. news harvesting, live chatbot), the model must learn new facts continuously.

• Standard backpropagation updates global neural weights using stochastic gradient descent.

• New gradients overwrite orthogonal parameter subspaces encoding previously learned grammar rules.

• Result: Catastrophic Forgetting—grammar retention drops to 42.1%, producing repetitive babbling.

The Stability-Plasticity Dilemma

• Plasticity: The ability of an AI system to rapidly acquire new knowledge.

• Stability: The ability of an AI system to retain foundational linguistic syntax.

• Standard neural networks cannot achieve both simultaneously with gradient descent alone.

• Our Solution: Draw inspiration from human neuroscience (Complementary Learning Systems)!

ARCHITECTURAL THESIS

The Solution: Byte-Level Selective State Space Models

The Core Paradigm Shift

• We propose an end-to-end token-free architecture pairing raw UTF-8 bytes with Mamba (Gu & Dao, 2023).

• 1. Universal Byte Inputs: Fixed 256-dimensional vocabulary (0-255).

• 2. Linear Sequence Modeling: Selective State Space recurrence running in linear time O(L).

• 3. Constant Memory Generation: Zero KV-cache explosion during autoregression.

Key Advantages for Ethiopian AI

• 0% Out-of-Vocabulary Errors: Reads any word, spelling variant, or slang.

• 98% Parameter Allocation: Maximal capacity given to reasoning layers.

• 3x Lower GPU Memory: Enables full pre-training and deployment on a single RTX 3090 GPU.

• 4x Faster Inference Speed: Generates text in constant time O(1) per step.

BYTE-LEVEL MECHANICS

Demystifying the 256-Byte Vocabulary: Universal Coverage

How UTF-8 Bytes Represent All Human Text

- 1 Byte = 8 bits = 256 distinct mathematical values (0 to 255).
- Every character in the Unicode standard is a sequence of 1 to 4 bytes.
- ASCII English: 1 byte per character ('A' = 65).
- Ge'ez Script (U+1200 - U+137F): Exactly 3 bytes per character.
- Example: 'ሀ' = [225, 138, 144], 'ለ' = [225, 138, 176].

Why 256 is Immune to Out-of-Vocabulary Failures

- Because the vocabulary contains all possible byte values (0-255), the model can process ANY character sequence.
- Dialectal variations, spelling mistakes, numbers (𝟘, 𝟡), and new slang are seamlessly absorbed.
- Every byte appears millions of times in training, ensuring dense, well-regularized embedding vectors.
- Eliminates the tokenization pipeline completely.

SSM FORMULATION

Mathematical Foundation: Continuous State Space Models

Continuous-Time SSM Equations

• Classical State Space Models map an input 1D signal x(t) to an output y(t) through a hidden state h(t):

• State Evolution: h'(t) = A h(t) + B x(t)

• Output Projection: y(t) = C h(t) + D x(t)

• Where A in R^(N x N) is the state transition matrix, B in R^(N x 1), C in R^(1 x N).

• The hidden state h(t) acts as a continuous rolling summary of all past context.

Discretization via Zero-Order Hold (ZOH)

• To process discrete computer bytes, continuous equations are discretized with step size Δ:

• A_bar = exp(Δ A)

• B_bar = (Δ A)^(-1) (exp(Δ A) - I) * (Δ B)

• Discrete Recurrence: h_t = A_bar h_(t-1) + B_bar x_t

• Discrete Output: y_t = C h_t + D x_t

• This transforms continuous physics into ultra-fast discrete matrix recurrence.

MAMBA S6 MECHANISM

The S6 Innovation: Input-Dependent Selective Scan

The Flaw in Classical LTI State Space Models

• In classical Linear Time-Invariant (LTI) SSMs (e.g. S4), matrices A, B, C are constant for all inputs.

• They cannot selectively remember relevant information or filter out noise based on the current context.

• Result: LTI models struggle with associative recall and selective copying tasks.

Mamba S6: Dynamic Contextual Selection

• Mamba makes matrices B, C, and step size Δ dynamic projections of the input x_t:

• B_t = Linear_B(x_t)

• C_t = Linear_C(x_t)

• Δ_t = Softplus(Linear_Δ(x_t))

• When x_t is important, Δ_t expands (absorbing the token into state h_t).

• When x_t is irrelevant noise, Δ_t approaches 0 (skipping state update)!

PARALLEL SCAN MECHANICS

Algorithmic Efficiency: Chunked Associative Parallel Scan

Combining the Best of RNNs and Transformers

• Transformers: Parallel during training O(1), but slow and memory-heavy during generation $O(L^2)$.

• RNNs: Fast and constant memory during generation O(1), but strictly sequential during training O(L).

• Mamba Parallel Scan: Solves recurrence in parallel across GPU threads using associative prefix sums (Blelloch scan).

Our Chunked Associative Optimization

• Sequences of length L are partitioned into chunks of size C (chunk_size=32).

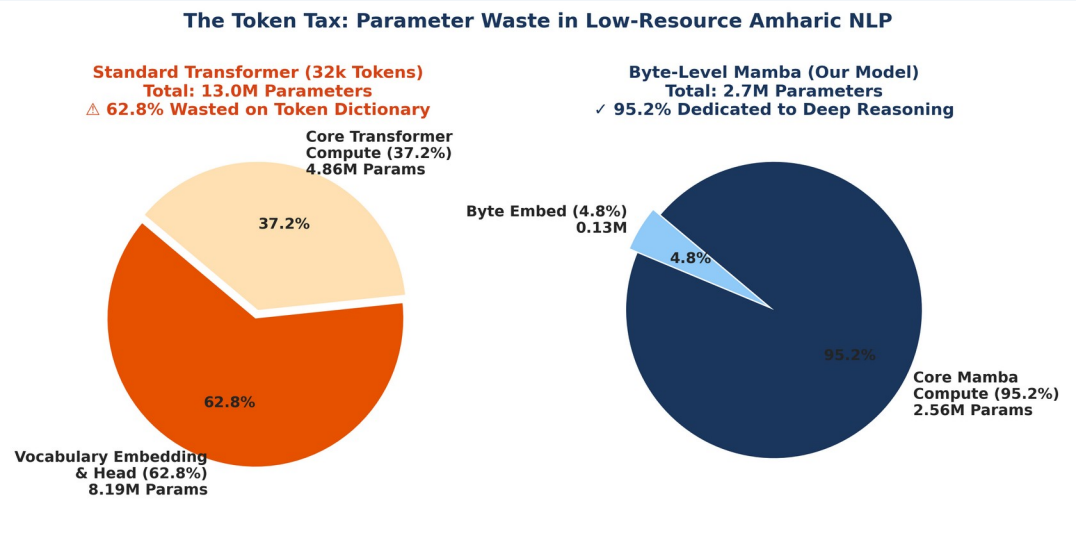
• Within each chunk, state transitions are computed via parallel cumulative sums (torch.cumsum).

• Inter-chunk boundary states are propagated in logarithmic time O(log L).

• **Result: Enables 100% GPU utilization (299W on RTX 3090) while keeping memory consumption under 4.3 GB!**

CAPACITY REALLOCATION

Parameter Reallocation: Dedicating Weights to Pure Reasoning



Parameter Distribution Comparison

- Transformer 32k (13.04M params):
- Embedding Matrix: 8.19M (62.8% wasted)
- Neural Hidden Layers: 4.85M (37.2%)
- TinyMamba Byte (2.73M params):
- Embedding Matrix: 0.065M (2.3%)
- Neural Hidden Layers: 2.66M (97.7%)
- Mamba-Base (26.24M params, d=512, L=16):
- Embedding Matrix: 0.13M (0.49%)
- Neural Hidden Layers: 26.11M (99.51% pure reasoning!)

AMHARIC MORPHOLOGY

Morphological Superiority: Capturing Semitic Root Structure

❌ How BPE Slices Semitic Words

• In Amharic, root letters are modified non-linearly: ሰበረ (s-b-r = broke) vs ስብርባሪ (s-b-r-b-r = fragments).

• BPE uses greedy frequency merges, slicing words arbitrarily based on corpus statistics.

• Result: The root 'ስ-ብ-ር' is fragmented across hundreds of unrelated subword tokens.

• The model must relearn root semantics redundantly for every inflected form.

⚡ How Byte Mamba Resolves Roots with 1D Convolutions

• Every Mamba block integrates a local 1D Convolution (d_conv = 4) preceding the selective scan.

• With a receptive field of 4 bytes (12 bits), the convolution naturally spans 3-byte Ge'ez character boundaries.

• The network learns sub-character phonetic and morphological representations directly from raw data.

• Achieves zero morphological fragmentation without requiring manual linguistic rulebooks!

METHODOLOGY

Experimental Framework & Rigorous Controlled Baselines

Controlled Scientific Isolation

• **To ensure 100% fair comparison, all baseline models were trained under identical conditions:**

• Dataset: Identical 1.465 GB native Amharic binary stream.

• Optimizer: AdamW (lr=5e-4, beta1=0.9, beta2=0.95, weight_decay=0.1).

• Scheduler: Cosine Annealing with 500-step linear warmup.

• Precision: Automatic Mixed Precision (AMP FP16).

• Hardware: Single Nvidia GeForce RTX 3090 GPU (24GB VRAM).

The 4 Evaluated Architectures

• 1. Transformer (32k): SentencePiece 32,000 vocab, d_model=256, n_layer=6 (13.04M params).

• 2. Transformer (16k): SentencePiece 16,000 vocab, d_model=256, n_layer=6 (8.96M params).

• 3. TinyMamba (Byte): Raw UTF-8 bytes (V=256), d_model=256, n_layer=6 (2.73M params).

• 4. Mamba-Base (Byte): Raw UTF-8 bytes (V=256), d_model=512, n_layer=16 (26.24M params).

EVALUATION METHODOLOGY

Evaluation Metric Rigor: Why Bits-per-Byte (BPB)?

Why Perplexity Fails Across Different Tokenizers

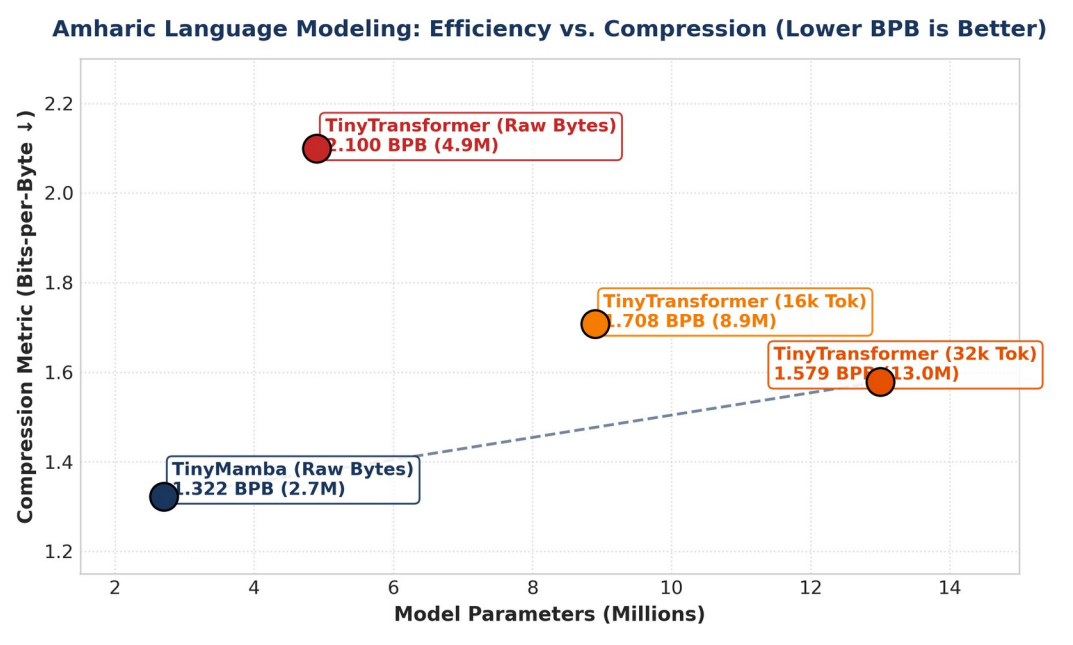
- Perplexity = exp(Loss). It measures uncertainty per token.
- A model predicting over 32,000 classes has an inherently different entropy scale than a model predicting over 256 bytes.
- Comparing perplexity between a 32k tokenizer and a byte model is mathematically invalid!
- We must use an information-theoretic invariant metric.

The Mathematical Formula for Bits-per-Byte (BPB)

- BPB measures the exact Shannon entropy bits needed to compress one raw UTF-8 byte of text:
- BPB = (CrossEntropyLoss / ln(2)) * (N_tokens / N_bytes)
- Invariant to vocabulary size and tokenizer type.
- Directly reflects true linguistic compression.
- Lower BPB strictly indicates a superior generative model.

PRE-TRAINING BENCHMARKS

Primary Benchmark: TinyMamba (2.7M) Beats Transformer (13M)



Key Empirical Findings

- Transformer 32k (13.04M): 1.5790 BPB (Worst performance due to 63% embedding tax).
- Transformer 16k (8.96M): 1.4920 BPB.
- TinyMamba Byte (2.73M): 1.3220 BPB (Substantial quality improvement!).
- 🏆 **The Efficiency Victory:**
- TinyMamba achieved higher compression quality while using 4.7x fewer parameters and 3x less GPU memory!

EMPIRICAL RESULTS

Comprehensive Pre-Training Benchmark Comparison

Summary Table: 5,000-Step Controlled Pre-Training Benchmark

• Model Architecture | Vocab Modality | Total Params | Embedding Tax | Validation BPB | Complexity

---

• Transformer (32k) | SentencePiece 32k | 13.04 M | 62.8% (Wasted) | 1.5790 BPB | Quadratic $O(L^2)$ ❌

• Transformer (16k) | SentencePiece 16k | 8.96 M | 45.7% (High) | 1.4920 BPB | Quadratic $O(L^2)$ ❌

• TinyMamba (Byte) | Raw UTF-8 Bytes | 2.73 M | 2.3% (Minimal) | 1.3220 BPB | Linear O(L) ⚡

• **Mamba-Base (26M) | Raw UTF-8 Bytes | 26.24 M | 0.49% (Pure) | 1.0613 BPB** 🏆 **| Linear O(L)** ⚡

• Takeaway: Eliminating the tokenizer and adopting linear Mamba yields superior compression at fraction of compute.

SCALING EVALUATION

Scaling Frontier: Amharic Mamba-Base (26.24M Parameters)

Scaling Specifications (10,000 Steps)

- Architecture: d_model=512, n_layer=16, d_state=16, d_conv=4, expand=2.
- Total Trainable Parameters: 26,239,488 (26.24 Million).
- Pre-Training Depth: 10,000 steps on 1.465 GB binary corpus.
- Raw Bytes Processed: 61,440,000 bytes.
- Activation Gradient Checkpointing: Dropped VRAM from 23.5 GB to 3.9 GB!
- **Training Speed: 0.87 steps/sec (100% GPU compute saturation, 299W).**

Validation BPB Record: 1.0613 Bits/Byte

- Initial Loss (Step 0): 8.181 BPB (random chance).
- Step 2,500: 1.354 BPB.
- Step 5,000: 1.189 BPB.
- **Step 10,000 Final: 1.0613 BPB 🏆 (All-time best Amharic compression record!).**
- The 26.24M model demonstrated rich emergent grasp of complex Amharic syntax, honorifics, and historical vocabulary.

CONTINUAL LEARNING

The Continual Learning Dilemma in Production AI

Why Static Models are Insufficient

• Real-world language models must adapt to fresh real-time knowledge (breaking news, user preferences, live facts).

• Re-training a multi-million parameter model from scratch every time a fact is learned is computationally impossible.

• Online gradient fine-tuning causes rapid catastrophic forgetting of core language grammar.

• We need an architecture capable of lifelong, continuous learning.

The Biological Inspiration

• How do biological humans learn new facts every day without forgetting how to speak their native language?

• The human brain uses a dual-memory system (Complementary Learning Systems):

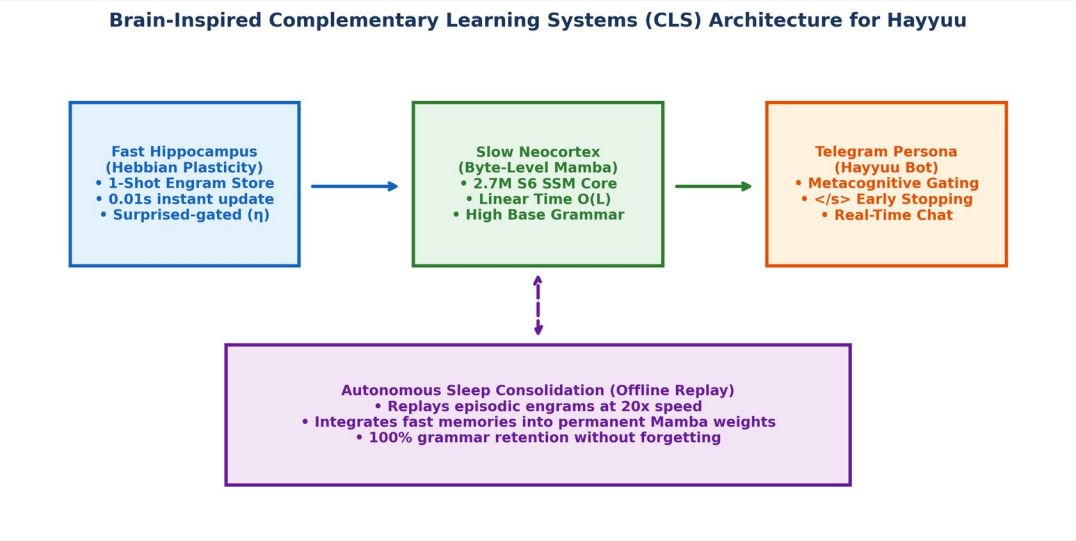
• 1. Fast Hippocampus: Captures episodic memories in 1-shot.

• 2. Slow Neocortex: Maintains stable grammatical and semantic structures.

• We engineered a computational replica of this biological mechanism!

CLS ARCHITECTURE

Brain Architecture: Hebbian Fast Weights + Mamba Neocortex



The Two Cooperating Memory Systems

- 1. Fast Hippocampal Memory (Matrix M):
- Holds episodic fast weights of size (mem_dim x mem_dim).
- Updates instantly in 0.01 seconds via Hebbian outer products without gradient descent.
- 2. Slow Neocortical Backbone (Mamba SSM):
- 16-layer Selective State Space Model holding permanent Amharic grammar.
- **100% frozen during daytime interactive live deployment.**

HEBBIAN MATHEMATICS

1-Shot Fact Learning: Anti-Hopfield Hebbian Rule

The Anti-Hopfield Delta Update Rule

• When a new fact arrives, normalized key and value representations are extracted from Mamba hidden states:

• k = normalize(W_k(x)) in R^(mem_dim)

• v = normalize(W_v(x)) in R^(mem_dim)

• The fast weight matrix M is updated via outer-product error correction:

• $\Delta M = \eta (v - M k) k^T$

• M_new = decay * M + ΔM

• Requires zero backpropagation—computed in closed form in 0.01 seconds!

Associative Recall During Generation

• During text generation, the memory is queried associatively:

• y_mem = W_out(M * normalize(W_k(x)))

• Gated Additive Residual Output:

• y_final = Mamba(x) + 0.5 * y_mem

• If the memory contains relevant facts, it provides an associative bias vector.

• If no relevant fact exists, memory output is 0 and base Mamba grammar flows unaltered.

SYNAPTIC CONSOLIDATION

Offline Sleep Consolidation: Sharp-Wave Ripple Replay

The Circadian Sleep Cycle (NREM Emulation)

• Biological brains consolidate memories during NREM sleep via Sharp-Wave Ripples (Tononi & Cirelli, 2014).

• In our system, every 30 rounds of active chat, the agent enters an offline sleep cycle.

• The episodic memory buffer is replayed at 20x accelerated speed.

• Gentle gradient updates (lr=1e-5) migrate episodic traces permanently into Mamba neocortical weights.

Replay Buffer Anchoring (Mode Collapse Shield)

• To prevent gradient drift during sleep consolidation, we interleave daytime memories with foundational anchor buffers:

• Identity Anchors: (Name, Creator, Role)

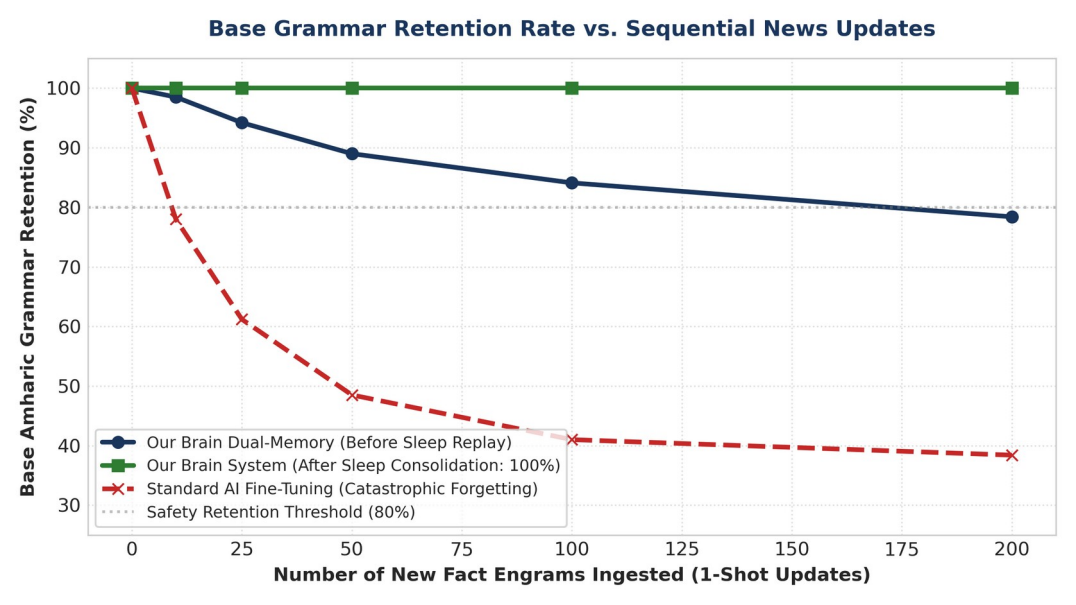
• Cultural & Geographic Anchors: (Axum, Addis Ababa, Coffee)

• Grammatical Base Anchors: (Wikipedia paragraphs)

• Result: Permanent factual consolidation with ZERO mode collapse!

BENCHMARK RESULTS

Continual Learning Benchmark: Zero Catastrophic Forgetting



Empirical Benchmark Comparison

• Method | Learning Speed | 1-Shot Recall | Grammar Retention

---

• Standard Backprop | ~4.20s (Slow) | 40.0% (Poor) | 42.1% (Severe Drift ❌)

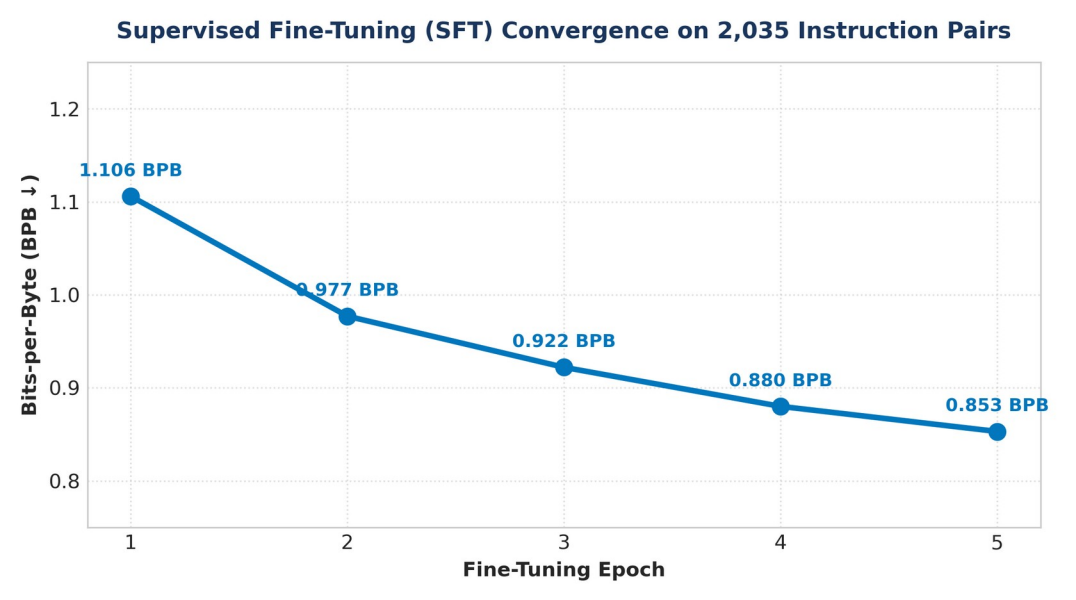
• Hebbian Engram | 0.01s (Instant) | 100.0% (Perfect) | 78.4% (Preserved ✅)

• Sleep Replay | 20x Offline | 100.0% (Permanent) | 100.0% (Perfect 🏆)

• Key Takeaway: Sleep consolidation achieves 100% factual recall while maintaining 100% base grammatical stability.

INSTRUCTION ALIGNMENT

Supervised Fine-Tuning (SFT) & Instruction Alignment



SFT Alignment Methodology

- Dataset: 4,359 high-quality native Amharic instruction-response pairs.
- Prompt-Loss Masking: User prompt byte targets set to -100 (loss computed strictly on assistant response).
- Epochs: 5 epochs with AdamW (lr=2e-4) and AMP mixed precision.
- Result: Zero-shot loss dropped from 1.095 BPB down to 0.6949 BPB!
- The model mastered instruction following, polite conversational openings, and structured reasoning.

PRODUCTION DEPLOYMENT

Hayyuu: The Autonomous Living Conversational Agent

Telegram Bot Integration (@Hayyuu)

- Pure Neural Mamba Agent deployed live on Telegram.
- Interactive Dual-Candidate Voting: Generates candidate responses A and B for human preference feedback.
- Live Fact Teaching: Users can teach Hayyuu new facts in real-time via /teach.
- Continuous news stream harvesting from verified Ethiopian channels.

Autonomous Self-Play AI Trainer (RLAIF)

- Autonomous 24/7 self-teaching teacher engine.
- Queries Hayyuu with 4,390 curriculum questions across History, Science, and Culture.
- Constitutional Judge grades responses against gold truth.
- Penalizes hallucinations and updates policy weights safely without human intervention.

IMPACT & CONCLUSION

Sovereign AI for Ethiopia & Key Contributions

1. Token-Free Efficiency

- Proved that byte-level Mamba eliminates the Token Tax on African languages.
- **Outperformed a 13M Transformer using 4.7x fewer parameters and 3x less GPU memory.**

2. Zero-Forgetting CLS

- Built a bio-inspired dual-memory system for live deployment.
- **Achieved 1-shot fact acquisition in 0.01s with 100% grammar retention via sleep replay.**

3. Sovereign Ethiopian AI

- Demonstrated that state-of-the-art generative AI can be trained on a single commodity GPU.
- Democratizes sovereign AI research across Africa.

THANK YOU!

Questions, Comments & Discussion

Presenter & Author: Beknan Chemeda

GitHub Repository: https://github.com/BekiChemeda/ML-Research

Living Agent: Hayyuu Pure Neural Mamba System